



Estrutura de Dados não Lineares

Prof. Thiago Koman Schmidt



Planejamento da aula

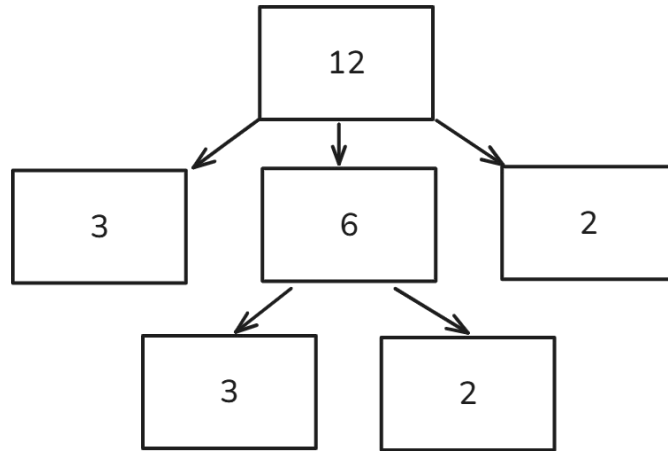
- Árvores genéricas
- Árvores binárias
- Aplicações e casos de uso



Árvores genéricas

O que são Árvores Genéricas?

- Cada nó pode ter qualquer quantidade de filhos.
- Flexível para representar estruturas sem limite fixo (e.g. HTML DOM, arquivos e pastas).



Representação: ponteiros para lista de filhos ou lista encadeada.

Propriedades das Árvores Genéricas

Grau de um nó: número de filhos diretos

Grau da árvore: maior grau entre os nós

****Mais complexas para percorrer em comparação com árvores binárias**

Vantagens:

- Estrutura altamente flexível
- Ideal para casos onde o número de filhos varia

Desvantagens:

- Percursos são mais complexos
- Menor eficiência para certas operações

Aplicações das Árvore Genéricas

- Representação de menus em interfaces

Agrupamento por expansão

Agrupamento do Menu 1	
Agrupamento do Menu 2	
Agrupamento do Menu 3	
Agrupamento do Menu 4	

Agrupamento por rótulos

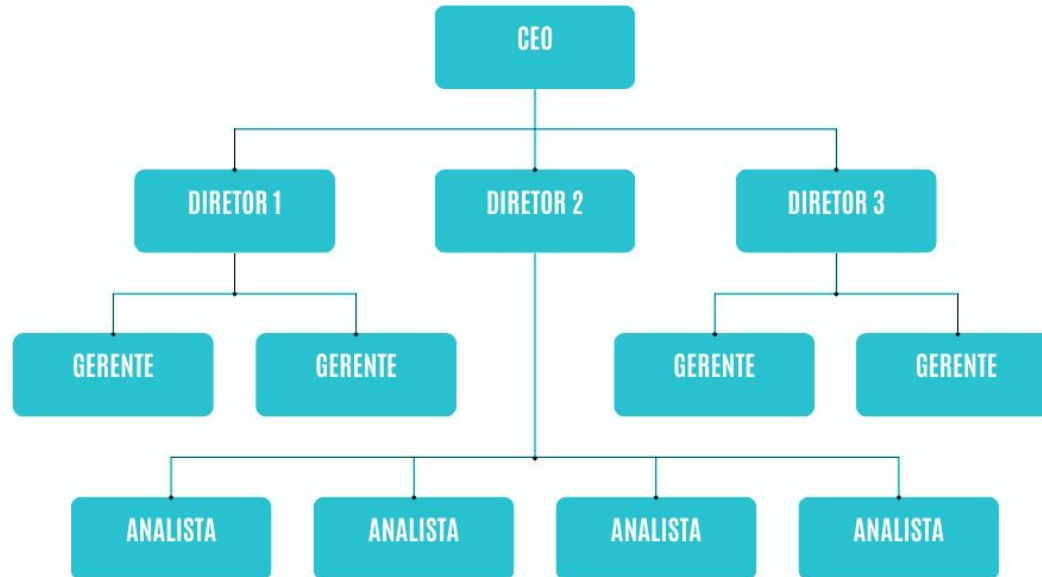
Agrupamento do Menu 1	
Item do Menu 1	
Item do Menu 2	
Item do Menu 3	
Item do Menu 4	
Agrupamento do Menu 2	
Agrupamento do Menu 3	
Agrupamento do Menu 4	

Agrupamento por dividers

Item do Menu 1	
Item do Menu 2	
Item do Menu 3	
Item do Menu 4	
Item do Menu 5	
Item do Menu 6	

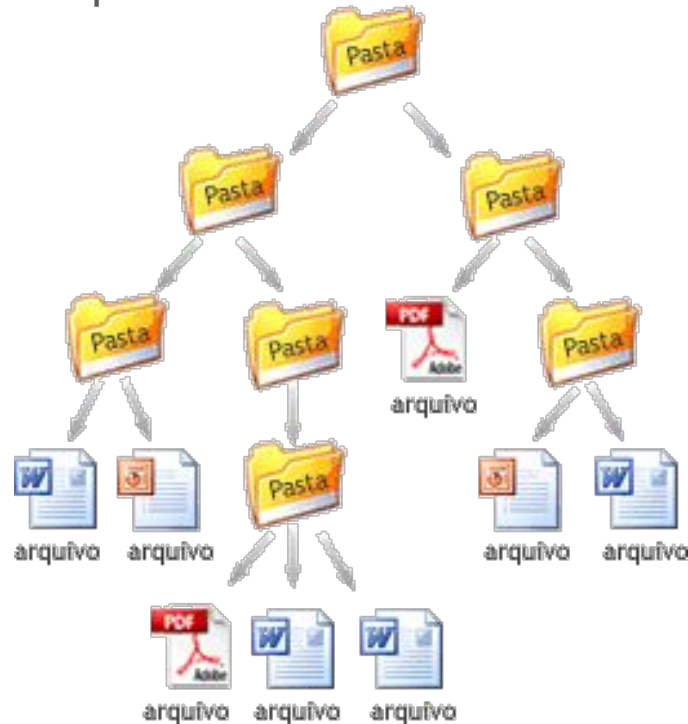
Aplicações das Árvore Genéricas

- Hierarquias organizacionais



Aplicações das Árvore Genéricas

- Sistemas de arquivos e pastas



Aplicações das Árvores Genéricas

- DOM (Document Object Model) em navegadores

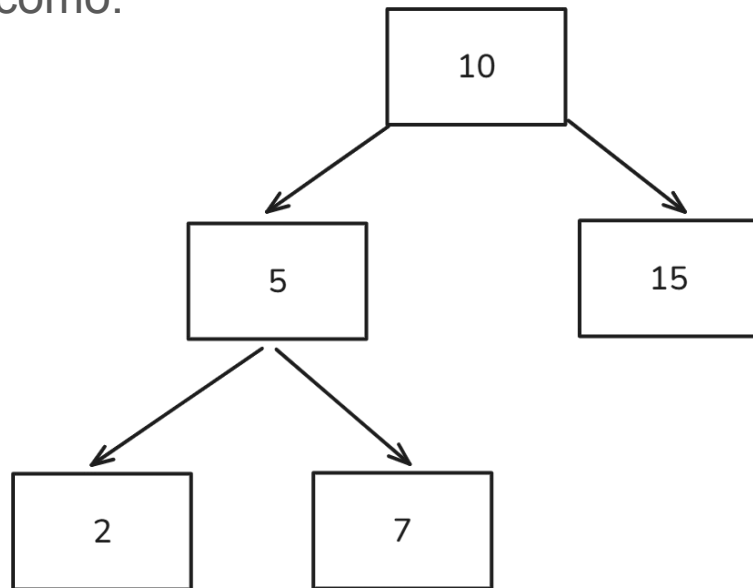
```
▼ [[Prototype]]: Window
  PERSISTENT: 1
  TEMPORARY: 0
  ▶ constructor: f Window()
  Symbol(Symbol.toStringTag): "Window"
  ▼ [[Prototype]]: WindowProperties
    Symbol(Symbol.toStringTag): "WindowProperties"
    ▼ [[Prototype]]: EventTarget
      ▶ addEventListener: f addEventListener()
      ▶ dispatchEvent: f dispatchEvent()
      ▶ removeEventListener: f removeEventListener()
      ▶ when: f when()
      ▶ constructor: f EventTarget()
      Symbol(Symbol.toStringTag): "EventTarget"
      ▼ [[Prototype]]: Object
        ▶ constructor: f Object()
        ▶ hasOwnProperty: f hasOwnProperty()
        ▶ isPrototypeOf: f isPrototypeOf()
        ▶ propertyIsEnumerable: f propertyIsEnumerable()
        ▶ toLocaleString: f toLocaleString()
        ▶ toString: f toString()
        ▶ valueOf: f valueOf()
        ▶ __defineGetter__: f __defineGetter__()
        ▶ __defineSetter__: f __defineSetter__()
        ▶ __lookupGetter__: f __lookupGetter__()
        ▶ __lookupSetter__: f __lookupSetter__()
        __proto__: (...)
        ▶ get __proto__: f __proto__()
        ▶ set __proto__: f __proto__()
```



Árvores binárias

Propriedades das Árvores Binárias

- Cada nó tem no máximo dois filhos: esquerdo e direito
- Muito utilizadas em algoritmos e estruturas como:
 - Árvores de expressão
 - Índices de banco de dados (AVL, B-trees)
 - Árvores de busca binária (BST)



Propriedades das Árvores Binárias

Ordem importa: filho esquerdo $\neq$ filho direito

Tipos comuns:

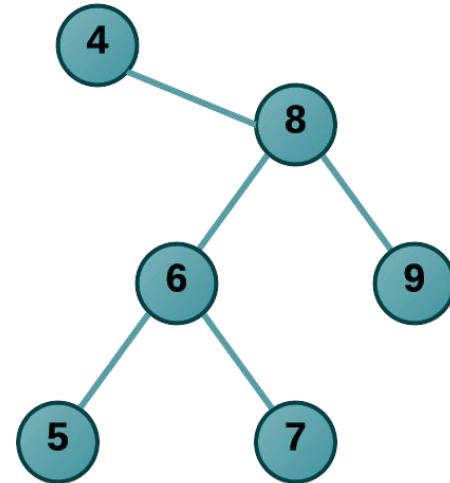
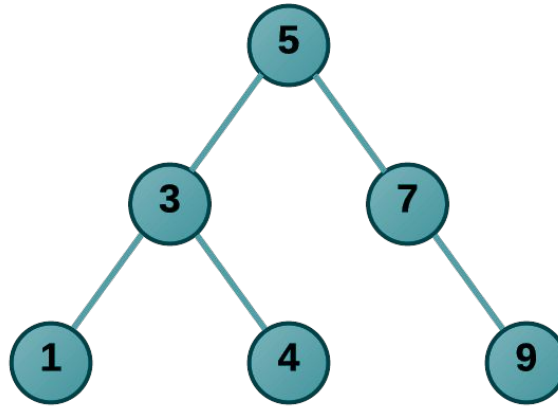
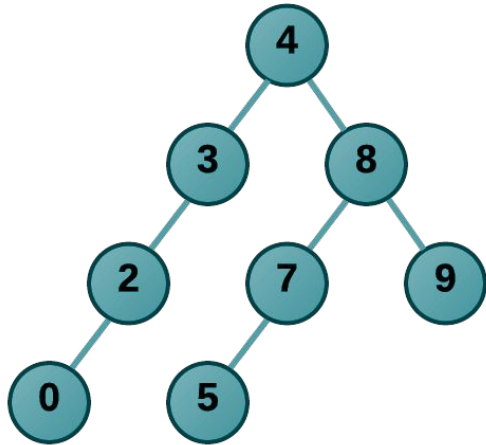
- **Completas:** todos os nós preenchidos até o penúltimo nível
- **Cheias:** todos os nós têm 0 ou 2 filhos
- **Balanceadas:** alturas das subárvores esquerda e direita são semelhantes

Fórmulas importantes:

- Número máximo de nós em nível (n): 2^n
- Altura mínima de uma árvore binária com n nós: $\log_2(n + 1)$

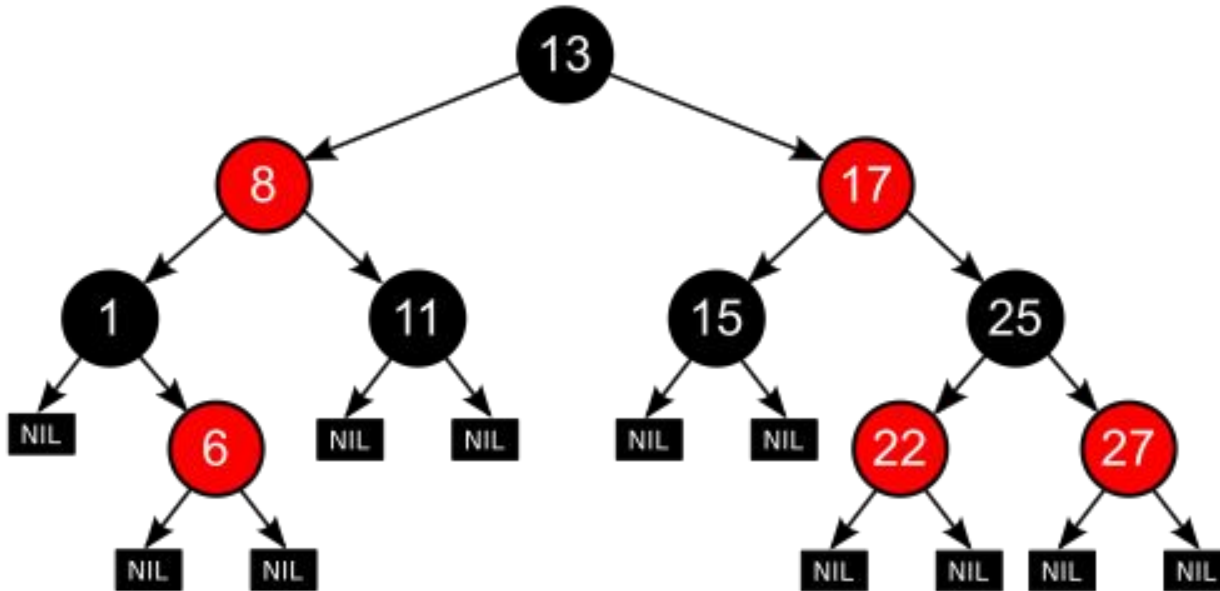
Aplicações das Árvores Binárias

- Árvores de busca binária (BST): buscas rápidas



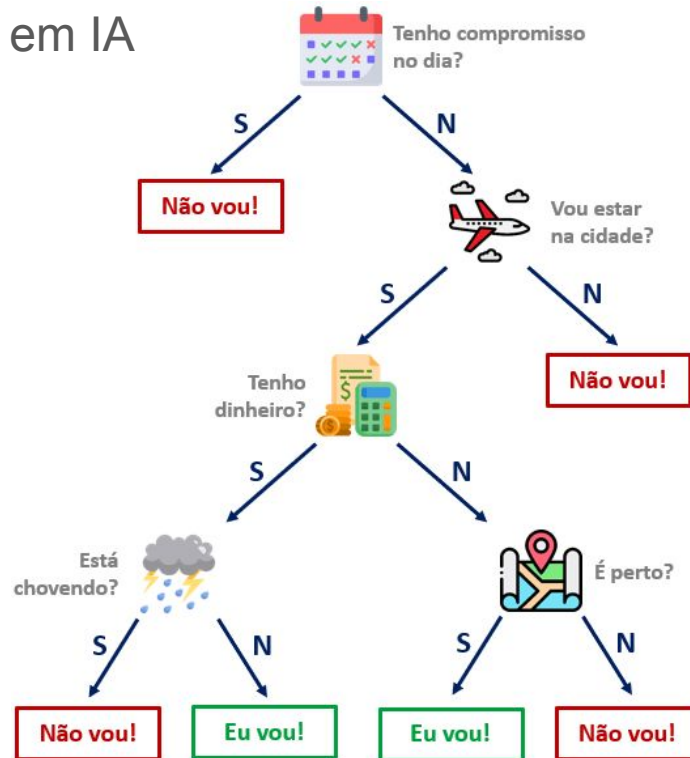
Aplicações das Árvores Binárias

- Árvores AVL e Red-Black: balanceadas para alta performance



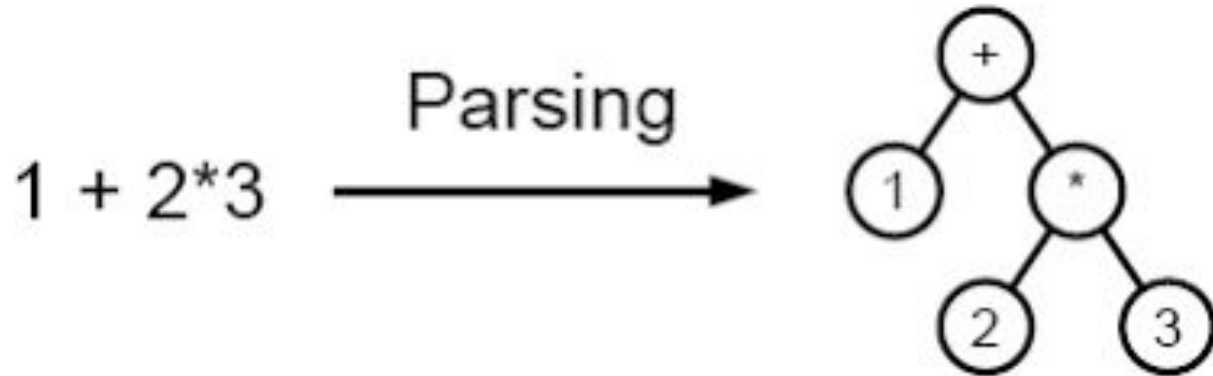
Aplicações das Árvore Binárias

- Árvores de decisão em IA



Aplicações das Árvores Binárias

- Parsing de expressões matemáticas e linguagens





Conversão

Técnicas de conversão

Árvore Genérica (ou N-ária): cada nó pode ter N filhos.

Árvore Binária: cada nó pode ter no máximo 2 filhos (esquerdo e direito).

Problema: como representar uma árvore genérica dentro da estrutura de árvore binária?

Técnicas mais usadas:

- Conversão Filho Mais à Esquerda – Irmão à Direita (Left Child – Right Sibling).
- Conversão via Lista de Filhos.
- Conversão via Recursão direta (aproveitando ponteiros binários).

Técnica 1: Filho Mais à Esquerda – Irmão à Direita

Cada nó da árvore genérica é representado em uma árvore binária:

Ponteiro Esquerdo → primeiro filho.

Ponteiro Direito → próximo irmão.

Permite representar qualquer árvore genérica como binária.

```
struct Node {  
    int data;  
    Node* left;    // primeiro filho  
    Node* right;   // próximo irmão  
    Node(int val) : data(val), left(NULL), right(NULL) {}  
};
```

Técnica 2: Conversão via Lista de Filhos

Cada nó mantém uma lista dinâmica de filhos, mas ainda pode ser mapeada em árvore binária.

Estrutura híbrida: usa ponteiro para primeiro filho + lista encadeada de irmãos.

Mais flexível, mas exige ponteiros adicionais.

```
struct GenNode {  
    int data;  
    vector<GenNode*> filhos; // genérica  
};  
  
struct BinNode {  
    int data;  
    BinNode* left; // primeiro filho  
    BinNode* right; // próximo irmão  
};
```

Técnica 3: Conversão Recursiva

- Estratégia recursiva:
 - O primeiro filho vira o ponteiro esquerdo.
 - Cada irmão subsequente vira o ponteiro direito do anterior.
- Implementação recursiva facilita conversão de uma árvore N-ária → Binária.

```
BinNode* convert(GenNode* root) {  
    if (!root) return NULL;  
    BinNode* bRoot = new BinNode{root->data, NULL, NULL};  
    if (!root->filhos.empty()) {  
        bRoot->left = convert(root->filhos[0]);  
        BinNode* temp = bRoot->left;  
        for (int i = 1; i < root->filhos.size(); i++) {  
            temp->right = convert(root->filhos[i]);  
            temp = temp->right;  
        }  
    }  
    return bRoot;  
}
```



Atividade

Atividade

Vamos montar uma árvore genealógica!

1. **Construa sua árvore genealógica** usando o conceito de **árvore genérica**, representando ao menos **três gerações** (ex: avós → pais/tios → você/irmãos/primos).
2. Represente cada “casal/solteiro” como um nó, e foque em um ascendente comum.
3. Em seguida, **utilize a técnica "Filho Mais à Esquerda – Irmão à Direita"** para **converter sua árvore genealógica em uma árvore binária**.

Dica: desenhe a árvore genérica primeiro e depois represente a mesma estrutura em binária, conectando o primeiro filho como filho esquerdo e os irmãos como filhos direitos.

DÚVIDAS?

